

Test-Writing Learnings

A reference for writing more tests in this codebase yourself. Reads top-to-bottom — pure beginner-to-pattern flow.

1. The one rule

When testing a component or page, replace every `src/features/**` module it imports with `vi.mock(...)`. The component renders for real; its hooks return whatever you tell them to.

That means **no MSW, no fetch interception, no Supabase test container**. Mock the hook, not the network. The price you pay is that you have to model the hook's return shape (`{ isPending, isError, data, mutate, ... }`) — but the same handful of shapes covers everything, and we built factories for them.

For testing the hooks themselves (the ~5 non-trivial ones), mock one level deeper: replace `lib/supabase/client.js` and let the real hook run.

2. Vitest functions you'll use a lot

`vi.fn()` — the mock function

Creates a mock function. Records every call, lets you configure what it returns.

```
const fn = vi.fn()
fn('a', 'b')
expect(fn).toHaveBeenCalledWith('a', 'b')

fn.mockReturnValue(42)           // returns 42 on every call
fn.mockResolvedValue(42)        // returns Promise.resolve(42)
fn.mockRejectedValue(err)       // returns Promise.reject(err)
fn.mockImplementation(x => x*2) // custom behavior; you can read
    `this` if needed
fn.mockReturnValueOnce(7)       // only the next call returns 7, then
    falls through
```

`mockResolvedValue` is the one we used most — turns a sync mock into an async one in one line.

`vi.mock(path, factory)` — replace a whole module

The single most important Vitest feature for this codebase.

```
vi.mock('../features/auth/auth.jsx', () => ({
  useAuth: vi.fn(),
}))
```

Two things to know:

1. **Vitest hoists `vi.mock` calls to the top of the file**, above your imports. So the moment you import `{ LoginPage }` from `'./LoginPage.jsx'`, the import chain inside `LoginPage.jsx` already sees the mock — you don't need to do anything else.
2. **The factory runs *once*, when the module is first required**. You can't pass variables from your test into it directly (because of hoisting — your variables don't exist yet when the factory runs). That's the problem `vi.hoisted` solves.

`vi.hoisted(factory)` — variables that survive the hoist

This is the function you asked about. Here's the situation it fixes:

```
let authState = { session: null }           // declared at line 5

vi.mock('./features/auth/auth.jsx', () => ({
  useAuth: (selector) => selector(authState), // ✗ ReferenceError
}))
```

Even though `authState` appears textually before `vi.mock`, Vitest hoists the `vi.mock` call up — so when the factory runs, `authState` hasn't been declared yet. `ReferenceError`.

`vi.hoisted` is the escape hatch: **whatever you return from `vi.hoisted(...)` is hoisted alongside the `vi.mock` calls**, so the factory can see it.

```
const authRef = vi.hoisted(() => ({ current: { session: null } })))

vi.mock('./features/auth/auth.jsx', () => ({
  useAuth: (selector) => selector(authRef.current),
}))

// Later in your tests, mutate authRef.current to control what
// useAuth returns:
beforeEach(() => {
  authRef.current = { session: null, login: vi.fn() }
})
```

We use `authRef.current = { ... }` (mutating the property) so the reference inside the factory stays valid; the factory closes over the `authRef` object, not its `.current` value.

A second use of `vi.hoisted`: declaring a *bag of `vi.fn()` mocks* that you reference from many `vi.mock` factories.

```
const m = vi.hoisted(() => ({
  useGetNotes: vi.fn(),
  useCreateNote: vi.fn(),
  // ... etc
}))

vi.mock('./features/notes/note.js', () => ({
  useGetNotes: m.useGetNotes,
  useCreateNote: m.useCreateNote,
}))
```

Now `m.useGetNotes.mockReturnValue(...)` from any test sets the behavior. Same `vi.fn()` instance everywhere.

`vi.spyOn(obj, 'method')` — observe an existing function

When you want to know whether a method was called but don't want to replace its implementation:

```
const spy = vi.spyOn(queryClient, 'invalidateQueries')
// ... do stuff ...
expect(spy).toHaveBeenCalled({ queryKey: ['notes'] })
```

The original method still runs. Used in the optimistic-mutation and workspaces tests to assert react-query side-effects.

`vi.clearAllMocks` / `vi.resetAllMocks` / `vi.restoreAllMocks`

Easy to confuse:

Function	Clears <code>.mock.calls</code>	Resets <code>.mockImplementation</code>	Restores original
<code>clearAllMocks</code>	yes	no	no
<code>resetAllMocks</code>	yes	yes (to no-op)	no
<code>restoreAllMocks</code>	yes	yes	yes (for spies)

If your tests start failing in weird ways after adding new ones, suspect leftover `.mockReturnValue` from a previous test. Usually `clearAllMocks` is fine if every test sets its own returns; switch to `resetAllMocks` if you find yourself with stale implementations.

`vi.useFakeTimers()` / `vi.useRealTimers()`

Replaces `setTimeout`/`setInterval`/`Date.now`/etc. with controllable fakes.

```
beforeEach(() => { vi.useFakeTimers() })
afterEach(() => { vi.useRealTimers() })

it('does a thing after 2s', async () => {
  // ... trigger something that scheduled a setTimeout ...
  await act(async () => { vi.advanceTimersByTime(2000) })
  expect(stuff).toHaveBeenCalled()
})
```

Pitfall we hit twice: `vi.useFakeTimers()` combined with `userEvent` + `findBy*` / `waitFor` deadlocks. `user-event` uses timers for keystroke delays, and RTL's polling uses real `setTimeout`. When you fake them all, nothing advances. Workarounds: - For `setTimeout`-driven UI (3 s error clear, 1 s redirect delay): **use real timers** + `waitFor(..., { timeout: 4000 })`. Costs you wall-clock seconds in one test, saves you hours of debugging. - For tests with no `userEvent` and no RTL polling (like our channels teardown): fake timers work cleanly. Just remember to wrap `advanceTimersByTime` in `act(async () => ...)` to flush React updates.

`vi.resetModules()`

Drops the module cache so the next `await import(...)` re-evaluates the file from scratch. We used this for `channel.js` because it keeps a module-level CHANNELS map that would leak between tests.

```
let channelModule
beforeEach(async () => {
  vi.resetModules()
  channelModule = await import('./channel.js') // fresh module
})
```

`vi.mock` factories are re-applied on each fresh import — so the supabase mock stays in place. Only use this when a module genuinely has stateful top-level variables.

3. Testing Library functions you'll use a lot

`render(ui)` — mount a component

Returns a bunch of helpers, the most important being `unmount` (manually tear down — usually `afterEach(cleanup)` from `test/setup.js` handles it).

`screen.getByX / queryByX / findByX`

Three families, easy to mix up:

Prefix	If found	If not found
<code>getBy*</code>	returns element	throws
<code>queryBy*</code>	returns element	returns null
<code>findBy*</code>	returns Promise	rejects after timeout (~1 s default)

Rule of thumb: - `getBy*` when you expect the element to be present *right now* (synchronously). - `queryBy*` when you're **asserting the element is NOT there** (`expect(queryByText(...)).not.toBeInTheDocument()`). - `findBy*` when something will appear after an async step (mutation resolving, promise chain).

Which query to choose

In order of preference (Testing Library's official priority):

1. `getByRole('button', { name: /save/i })` — most realistic, also covers accessibility.
2. `getByLabelText(/email/i)` — for form inputs wrapped in a `<label>`.
3. `getByPlaceholderText(/title/i)` — for inputs without labels (we used this a lot, since the codebase rarely uses labels).
4. `getByText(/something/i)` — for non-interactive text content.
5. `getById('mdx-editor')` — last resort when nothing else fits (we used it for the MDX stub).

Multi-text-node gotcha: `getByText('author@x.com')` fails when the text is split across multiple text nodes in the same parent (e.g., `<p>Created by: <MemberEmail/></p>`). Use a `regex(/author@x\.com/)` — regexes match

against `textContent`, which includes all descendants.

userEvent — simulating user actions

Always prefer `userEvent` over `fireEvent` — `userEvent` simulates a real interaction (keystrokes, focus, etc.); `fireEvent` only fires a single DOM event.

```
import { userEvent } from '@testing-library/user-event'

await userEvent.type(screen.getByLabelText(/email/i), 'a@b.com')
await userEvent.click(screen.getByRole('button', { name: /submit/i
}))
```

`await` it — `userEvent` is `async` because real interactions involve multiple events with delays in between.

For tests that need both fake timers AND `userEvent` — see the pitfall in §2 above. Usually the answer is “don’t”.

renderHook(callback, { wrapper }) — testing hooks directly

```
const { result, rerender, unmount } = renderHook(
  () => useOptimisticMutation({ mutationFn, queryKey,
    optimisticApply }),
  { wrapper: ({ children }) => <QueryClientProvider client={qc}>
    {children}</QueryClientProvider> }
)

expect(result.current.isPending).toBe(false)
act(() => { result.current.mutate(1) })
await waitUntil(() => expect(result.current.isSuccess).toBe(true))
```

`result.current` is the hook’s latest return value. You can also pass `initialProps` and use `rerender(newProps)` to test prop changes:

```
const { rerender } = renderHook(
  ({ onP }) => useNoteChannel({ ..., onPresence: onP }),
  { initialProps: { onP: first } }
)
rerender({ onP: second })
```

act(callback) — flushing React updates

Wraps a callback that triggers React state updates so they finish before the next assertion. Two flavors:

```
act(() => { result.current.mutate(2) }) // sync work
await act(async () => { resolve({}) }) // async work
                                     (microtasks)
```

You usually don’t need it when calling `userEvent.*` (it wraps internally). You **do** need it when: - You call a captured callback directly (e.g., a broadcast event from a mocked channel) - You manually resolve a deferred promise that triggers a state update - You advance fake timers that fire `setTimeout` callbacks containing `setState`

If you see `Warning: An update to X inside a test was not wrapped in act(...)`, you missed one.

waitFor(callback, { timeout? }) — poll until success

Calls the callback repeatedly until it doesn't throw, or until timeout (default ~1 s). Use it for “thing should eventually appear”:

```
await waitFor(() => {
  expect(client.getQueryData(key)).toEqual([{ id: 1 }, { id: 3 }])
})
```

Pattern from the optimistic test that bit us once:

```
mutate() is fire-and-forget. The mutationFn doesn't run
synchronously — onMutate runs first (async), then mutationFn. If you
do act(() => mutate(1)) followed immediately by resolving a
deferred mutationFn, your resolver is still undefined. Wait for the
mock to be called before driving its promise:
```

```
act(() => { result.current.mutate(1) })
await waitFor(() => expect(mutationFn).toHaveBeenCalled())
await act(async () => { resolve({}) }) // now safe
```

4. The patterns we built

4.1 Auth-style hoisted ref for selector hooks

useAuth(state => state.session) is a zustand store called with a selector. To control its return value across tests:

```
const authRef = vi.hoisted(() => ({ current: null }))

vi.mock('../features/auth/auth.jsx', () => ({
  useAuth: (selector) => selector(authRef.current),
}))

function setAuth(overrides = {}) {
  authRef.current = {
    session: null, loading: false,
    login: vi.fn(), logout: vi.fn(), signUp: vi.fn(),
    ...overrides,
  }
}

beforeEach(() => { setAuth() })

it('renders the redirect when logged in', () => {
  setAuth({ session: { user: { id: 'u1' } } })
  renderWithProviders(<LoginPage />)
  // ...
})
```

The mock function calls selector(authRef.current) *at call time*, so mutating authRef.current between tests works exactly as if you'd swapped the store's state.

4.2 Bag of hoisted mocks for page tests

When a page imports many feature modules, collect all the hooks into one hoisted object and mock each module with references to it:

```
const m = vi.hoisted(() => ({
  useGetNotes: vi.fn(), useDeleteNote: vi.fn(), useCreateNote:
    vi.fn(),
  useGetTagsForNotes: vi.fn(), useCurrentWorkspaceId: vi.fn(),
  useNoteChannel: vi.fn(), useCommandPalette: vi.fn(),
  // ... etc
}))

vi.mock('../features/notes/note.js', () => ({
  useGetNotes: m.useGetNotes,
  useDeleteNote: m.useDeleteNote,
  useCreateNote: m.useCreateNote,
  QUERY_KEYS: { list_in_folder_all: (...a) => ['notes', ...a,
    'list'] },
  PAGE_SIZE: 10,
}))

vi.mock('../features/tags/tags.js', () => ({ useGetTagsForNotes:
  m.useGetTagsForNotes }))
// ... etc
```

Per-test, just set whatever returns you need:

`m.useGetNotes.mockReturnValue(success([...])).` Scales cleanly to 6+ feature modules in `Notes.test.jsx`.

4.3 Query-state and mutation factories

The same response shapes show up everywhere. Build factories:

```
const success = (data) => ({ isPending: false, isLoading: false,
  isError: false, isSuccess: true, data, error: null })
const pending = () => ({ isPending: true, isLoading: true,
  isError: false, isSuccess: false, data: undefined, error:
    null })
const errored = (e = new Error('boom')) => ({ isPending: false,
  isLoading: false, isError: true, isSuccess: false, data:
    undefined, error: e })

const mutation = (overrides = {}) => ({
  mutate: vi.fn(), mutateAsync: vi.fn(), reset: vi.fn(),
  isPending: false, isError: false, isSuccess: false, isIdle:
    true,
  data: undefined, error: null,
  ...overrides,
})
```

Then in tests:

```
m.useGetNotes.mockReturnValue(success([note1, note2]))
m.useDeleteNote.mockReturnValue(mutation({ isPending: true })))
```

When you're ready, promote these to `test/Utils/queryStates.js`.

4.4 Swappable supabase mock via a getter

For hook-internal tests, the consuming module imports supabase from `lib/supabase/client.js`. You want a *fresh* supabase mock per test. The hoisted-ref-of-mock pattern:

```
const supabaseRef = vi.hoisted(() => ({ current: null }))

vi.mock('../lib/supabase/client.js', () => ({
  get supabase() { return supabaseRef.current },
}))

beforeEach(() => {
  supabaseRef.current = makeSupabaseMock()
})
```

The `get` (a getter) is key — it re-evaluates every time the consuming module reads `supabase`, so swapping `supabaseRef.current` is enough.

4.5 Chainable supabase stub with response queues

For `await supabase.from('x').select().eq(...)`, you need a thenable builder that resolves to canned data:

```
function makeSupabaseMock() {
  let chainQueue = []
  let rpcQueue = []
  const pop = (q) => q.shift() ?? { data: null, error: null }

  const builder = {}
  ;
  ['from', 'select', 'eq', 'match', 'is', 'ilike', 'range', 'insert', 'delete', 'update']
    .forEach(m => { builder[m] = vi.fn(() => builder) })
  builder.then = (resolve, reject) =>
    Promise.resolve(pop(chainQueue)).then(resolve, reject)

  return {
    from: builder.from,
    rpc: vi.fn(() => Promise.resolve(pop(rpcQueue))),
    auth: { onAuthStateChange: vi.fn(() => ({ data: {
      subscription: { unsubscribe: vi.fn() } } } )) },
    channel: vi.fn(),
    removeChannel: vi.fn(() => Promise.resolve()),
    _enqueue: (r) => chainQueue.push(r),
    _enqueueRpc: (r) => rpcQueue.push(r),
  }
}
```

Test usage:

```
supabaseRef.current._enqueue({ data: [{id:1}, {id:2}], error: null
})
// the hook will `await supabase.from('foo').select()` → resolves to
that
```

Why a queue: if a single test triggers two `await` chains, you can line up two responses. The order matters — read your hook's code, count the `awaits`.

4.6 Capturing event-handler callbacks

When a component passes `onPresence` / `onBroadcast` / similar callbacks into a mocked hook, capture them so your test can drive events:

```
function installChannelCapture() {
  const channel = makeChannel()
  let broadcastCb = null
  m.useNoteChannel.mockImplementation(({ onBroadcast }) => {
    if (onBroadcast) broadcastCb = onBroadcast
    return { channel, isPending: false, isError: false,
      isSuccess: true }
  })
  return { channel, fireBroadcast: (p) => broadcastCb?.(channel,
    p) }
}

// in test:
const { fireBroadcast } = installChannelCapture()
renderEdit()
await act(async () => { fireBroadcast({ event: 'note_updated' }) })
expect(screen.getByText(/note changed
  remotely/i)).toBeInTheDocument()
```

When multiple components in the same tree call the same hook with different callbacks, filter by which prop is present (if `onBroadcast`) vs if `onPresence`)).

4.7 Optimistic mutation testing with a controlled promise

To observe optimistic state *mid-flight*, hold the `mutationFn` open and assert before resolving:

```
let resolveRpc
supabaseRef.current.rpc.mockImplementationOnce(() => new Promise(r
  => { resolveRpc = r }))

renderHook(() => useDeleteWorkspace(), { wrapper })
act(() => { result.current.mutate('w2') })

// Cache is now optimistically updated:
await waitFor(() => {
  expect(client.getQueryData(KEY)).toEqual([{id: 'w1'}, {id: 'w3'}])
})

// Now finish so react-query doesn't dangle:
await act(async () => { resolveRpc({ error: null }) })
await waitFor(() => expect(result.current.isSuccess).toBe(true))
```

4.8 Module-state isolation via `resetModules` + `dynamic import`

For files like `channel.js` with module-level `CHANNELS = {}`:

```
let channelModule
beforeEach(async () => {
  vi.resetModules()
  supabaseRef.current = makeSupabaseForChannels()
  channelModule = await import('./channel.js')
})
```

```
it('does a thing', () => {
  renderHook(() => channelModule.useNoteChannel({ ... }))
  // ...
})
```

Each test runs against a fresh copy of the module. `vi.mock` factories survive `resetModules`, so the supabase mock keeps working.

4.9 MDX editor stub via `forwardRef` + `useImperativeHandle`

`MDXEditor` uses a ref to expose `getMarkdown` / `setMarkdown`. The stub forwards a ref and exposes the same imperative API backed by local state:

```
vi.mock('@mdxeditor/editor', async () => {
  const React = await import('react')
  const MDXEditor = React.forwardRef(({ markdown, onChange }, ref)
    => {
    const [value, setValue] = React.useState(markdown ?? '')
    React.useImperativeHandle(ref, () => ({
      getMarkdown: () => value,
      setMarkdown: (v) => setValue(v ?? ''),
    }))
    return (
      <textarea
        data-testid="mdx-editor"
        value={value}
        onChange={(e) => { setValue(e.target.value);
        onChange?.(e.target.value) }}
      />
    )
  })
  return {
    MDXEditor,
    headingsPlugin: () => ({}),
    listsPlugin: () => ({}),
    quotePlugin: () => ({}),
    thematicBreakPlugin: () => ({}),
  }
})

vi.mock('@mdxeditor/editor/style.css', () => ({})) // CSS side-effect import
```

Now `ref.current.getMarkdown()` returns whatever was typed, `setMarkdown('x')` updates the rendered value — close enough to the real thing for behavior tests.

4.10 The shared providers wrapper

`test/utils/renderWithProviders.jsx` wraps in `QueryClientProvider` + `MemoryRouter`. Two modes:

```
// plain – for components that don't depend on URL state
renderWithProviders(<MyComponent />)

// with a routes table – for components reading useParams /
useSearchParams,
```

```
// or asserting redirects
renderWithProviders(null, {
  initialEntries: ['/notes/42/edit'],
  routes: [
    { path: '/notes/:id/edit', element: <EditNote /> },
    { path: '/', element: <div>HOME</div> }, //
    // redirect target sentinel
  ],
})
```

Doesn't support nested routes (for <Outlet/> components like ProtectedRoute). For those, drop down to inline <MemoryRouter><Routes>... </Routes></MemoryRouter>.

5. Pitfalls (and how to avoid them)

5.1 rerender loses the providers

```
const { rerender } = renderWithProviders(<NotesList />)
m.useGetNotes.mockReturnValue(success([...]))
rerender(<NotesList />) // x no router context anymore
```

RTL's rerender(ui) swaps the root tree but **loses any wrapping providers**. If you need to test multiple states of the same component, write **separate it() blocks**, each with its own render. Cheaper than fighting the API.

5.2 Three-banner tests need explicit unmount() between renders

When you render a component three times in one it() (pending / success / error), the rendered DOMs accumulate inside the same container until afterEach(cleanup) runs. getByText may find the wrong instance. Force a clean slate:

```
m.useCreateNote.mockReturnValue(mutation({ isPending: true }))
const { unmount: u1 } = renderWithProviders(<CreateNote />)
expect(screen.getByText(/creating/i)).toBeInTheDocument()
u1()

m.useCreateNote.mockReturnValue(mutation({ isSuccess: true }))
renderWithProviders(<CreateNote />)
expect(screen.getByText(/successfully/i)).toBeInTheDocument()
```

Or — better — just split into three it() blocks.

5.3 Persisted zustand state leaks

useWorkspaceStore uses zustand/middleware's persist → writes to localStorage on every change. Tests share localStorage. Reset in beforeEach:

```
beforeEach(() => {
  localStorage.clear()
  useWorkspaceStore.setState({ selectedId: null })
})
```

Same for any other persisted store you add.

5.4 React-Query retries make failing-query tests slow

Default retry is 3, with exponential backoff — tests that exercise the error path will take ~30 s before failing. Always set:

```
new QueryClient({
  defaultOptions: {
    queries: { retry: false },
    mutations: { retry: false },
  },
})
```

Or, if you're using the project's singleton client, override at the top of the test file:

```
import { queryClient } from '../../queryClient.js'

queryClient.setDefaultOptions({
  queries: { retry: false }, mutations: { retry: false },
})

beforeEach(() => { queryClient.clear() }) // drop cache between
                                         tests
```

5.5 <Outlet/> components need nested routes

<ProtectedRoute> returns <Outlet/>. Outlet renders the matched **child** route — which means you need a parent-child route structure, not the flat list our helper produces:

```
render(
  <MemoryRouter initialEntries={['/secret']}>
    <Routes>
      <Route element={<ProtectedRoute />}>
        <Route path="/secret" element={<div>SECRET
CONTENT</div>} />
      </Route>
    </Routes>
  </MemoryRouter>,
)
```

5.6 Mocking sibling page modules

If page A imports page B (e.g., Folders.jsx imports NotesList from Notes.jsx), don't recreate B's whole mock surface — stub the import instead:

```
vi.mock('./Notes.jsx', () => ({
  NotesList: () => <div data-testid="notes-list-stub" />,
}))
```

Keeps the test focused on the component under test.

5.7 The vi.mock factory is module-scope, not test-scope

Factories run once per module import. You **cannot** put `vi.mock` inside an `it()` and expect different behavior per test. To swap behavior per test: - Use `mockReturnValue` / `mockImplementation` on a hoisted `vi.fn()` - Or use the hoisted-ref pattern (§4.4)

5.8 `useEffect` initial-mount fires effects synchronously after render

Tests like “broadcasts on success” don’t need a click — just mount the component with `mockReturnValue(mutation({ isSuccess: true }))`. The effect that runs if `(isSuccess) channel.send(...)` fires on first render. `waitFor` until you see the call.

6. Skeleton for the next test you write

For a new page test:

```
import { screen, waitFor } from '@testing-library/react'
import { userEvent } from '@testing-library/user-event'
import { renderWithProviders } from
  '../test/utils/renderWithProviders.jsx'

const authRef = vi.hoisted(() => ({ current: null }))
const m = vi.hoisted(() => ({
  useThing: vi.fn(),
  // ... whichever hooks your page imports
}))

vi.mock('../features/auth/auth.jsx', () => ({
  useAuth: (selector) => selector(authRef.current),
}))
vi.mock('../features/things/thing.js', () => ({ useThing: m.useThing
  })))

import { ThingPage } from '../ThingPage.jsx'

const success = (data) => ({ isPending: false, isError: false,
  isSuccess: true, data, error: null })
const pending = () => ({ isPending: true, isError: false,
  isSuccess: false, data: undefined, error: null })
const mutation = (o = {}) => ({ mutate: vi.fn(), reset: vi.fn(),
  isPending: false, isError: false, isSuccess: false, ...o })

beforeEach(() => {
  vi.clearAllMocks()
  authRef.current = { session: null, login: vi.fn(), logout:
    vi.fn() }
})

describe('ThingPage', () => {
  it('shows loading', () => {
    m.useThing.mockReturnValue(pending())
    renderWithProviders(<ThingPage />)
    expect(screen.getByText(/loading/i)).toBeInTheDocument()
  })
})
```

```

it('handles click', async () => {
  const mut = mutation()
  m.useThing.mockReturnValue(success([{id: 1}]))
  // ... etc
  renderWithProviders(<ThingPage />)
  await userEvent.click(screen.getByRole('button', { name:
/go/i }))
  expect(mut.mutate).toHaveBeenCalledWith({id: 1})
})
})

```

For a new hook-internal test:

```

import { renderHook, waitFor, act } from '@testing-library/react'
import { QueryClientProvider } from '@tanstack/react-query'
import { makeSupabaseMock } from
'../../../../test/utils/mockSupabase.js'

const supabaseRef = vi.hoisted(() => ({ current: null }))

vi.mock('../../../../lib/supabase/client.js', () => ({
  get supabase() { return supabaseRef.current },
}))

import { useMyHook } from './myHook.js'
import { queryClient } from '../../../../queryClient.js'

queryClient.setDefaultOptions({ queries: { retry: false },
mutations: { retry: false } })

const wrapper = ({ children }) => (
  <QueryClientProvider client={queryClient}>{children}
</QueryClientProvider>
)

beforeEach(() => {
  queryClient.clear()
  supabaseRef.current = makeSupabaseMock()
})

it('fetches', async () => {
  supabaseRef.current._enqueue({ data: [{id:1}], error: null })
  const { result } = renderHook(() => useMyHook(), { wrapper })
  await waitFor(() => expect(result.current.isSuccess).toBe(true))
  expect(result.current.data).toEqual([{id:1}])
})

```

7. The decision tree

When you sit down to write a test, ask in order:

1. **Is the thing I'm testing a pure function?** Just import it and call it. No setup. (Examples: `debounce`, `normalize_key`, `keyMatches`.)
2. **Is it a zustand store with no async logic?** Render nothing — just call `useAuth.getState()`, `useAuth.setState(...)`. (Auth store methods.)
3. **Is it a React-Query hook?** `renderHook` + a real `QueryClient` via `wrapper`,

with supabase mocked. (Workspaces, optimistic.)

4. **Is it a React hook with effects but no react-query?** `renderHook` with whatever providers it needs. (`useChannel`, `useInitAuth`.)
5. **Is it a component or page?** `render` + `vi.mock` all the feature hooks it imports. Use the page-test skeleton.
6. **Does the component you're testing render a `<Navigate>`?** Use a routes table with a sentinel like `<div>HOME</div>` at the destination.
7. **Does the component have an `<Outlet/>`?** Inline `<MemoryRouter>`
`<Routes><Route element={...}><Route .../></Route></Routes>`
`</MemoryRouter>`.

If at any step you find yourself reaching for MSW or a real network call — stop and remember rule #1: mock at the hook boundary.